Convex Optimization with Limited Memory

Phillip Nicol
Harvard College
Cambridge, MA 02138
phillipnicol@college.harvard.edu

Pranay Tankala
Harvard College
Cambridge, MA 02138
pranay_tankala@college.harvard.edu

Abstract

In today's age of high-dimensional data, a clear understanding of the capabilities and limitations of memory-efficient learning algorithms is invaluable. Despite this, surprisingly little is known about the inherent complexity of memory-constrained optimization. In this paper, we work toward a solution to an open problem presented at COLT 2019 by Srebro and Woodworth [12], which asks for a characterization of the tradeoffs between the memory usage and first order oracle queries needed to optimize a convex Lipschitz function. We will attempt to prove upper bounds on the complexity of memory-constrained optimization by modifying existing optimization algorithms, and we will discuss a possible information theoretic approach to deriving a corresponding lower bound.

1 Introduction

To what extent do memory constraints impede efficient learning? This fundamental question was first posed in [9] by Shamir, who proved that for a simple statistical learning task called the 'hide-and-seek' problem, no algorithm with a small memory footprint can achieve optimal performance. Specifically, using information-theoretic techniques, [9] showed that any algorithm for the 'hide-and-seek' problem with a fixed number of memory bits must process more data samples than a constraint-free naïve algorithm in order to achieve comparable error.

Recently, there have been a number of exciting works proving even sharper memory-sample trade-offs for learning. For example, in [8], Raz proved that any algorithm for the task of parity learning using sub-quadratic memory would require *exponentially* more training samples than the standard, quadratic memory approach based on Gaussian elimination. The techniques developed by Raz for this problem, such as analyzing the 'significant vertices' of a branching program that models the learner, have since been adapted to more natural machine learning problems, including linear regression. In particular, the recent work of [10] showed that all streaming algorithms for linear regression fail to achieve optimal performance when using sub-quadratic memory.

In this paper, we study the complexity of memory-constrained convex optimization in an attempt to solve an open problem presented at COLT 2019 by Srebro and Woodworth [12].

1.1 Problem Setup

Let $F : \mathbb{R}^d \to \mathbb{R}$ be a convex, L-Lipschitz function. Suppose we'd like to solve the convex optimization problem

$$\min_{x \in \mathbb{R}^d : \|x\| \leq B} F(x) \tag{$\star$}$$

using an iterative first-order algorithm and only M bits of memory. If we were to use (sub)gradient descent, we'd need only $M = O(d \log LB/\varepsilon)$ bits of memory to minimize F up to error ε, and one can prove that this memory usage is optimal. Unfortunately, gradient descent is also known to take $O((LB/\varepsilon)^2)$ iterations to converge.

For CS281.

Alternatively, we could use the center-of-mass method, which uses quadratic memory (specifically, $O(d^2 \log^2(LB/\varepsilon))$ bits), but is known to converge after a mere $O(d \log LB/\varepsilon)$ iterations, which is also known to be optimal.

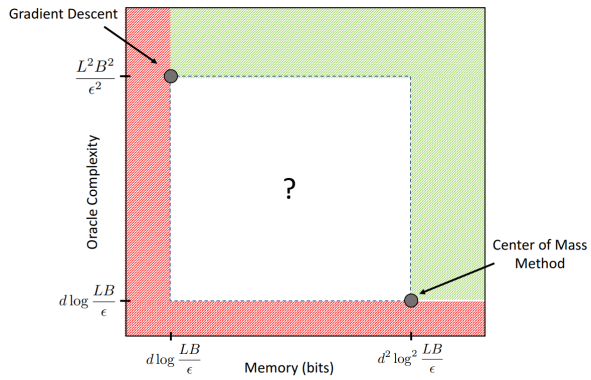

Figure 1: Memory vs. Oracle Complexity for Convex Optimization (taken from [12])

This interesting dichotomy can be seen in Figure 1, in which the horizontal axis measures memory usage, and the vertical axis, labeled "oracle complexity", measures the number of iterations involved in a first-order optimization algorithm. In this figure, the green region highlights the memory-query trade-offs that are known to be attainable with either gradient descent or the center-of-mass method, and the red region indicates trade-offs that are provably unattainable by any first-order algorithm.

Srebro and Woodworth pose the question of coloring the missing region in Figure 1: that is, completely characterizing the attainable trade-offs between memory usage and oracle complexity for convex optimization. In this paper, our goal is to begin to unravel this problem by attempting to prove the following two simpler claims, which deal only with the "corners" of the unknown region:

Conjecture 1. *There exists an iterative first-order convex optimization algorithm using $o(d^2 \log^2 LB/\varepsilon)$ memory and $o(L^2B^2/\varepsilon^2)$ iterations for the problem ($\star$).*

Conjecture 2. *No iterative first-order convex optimization algorithm using $O(d \log LB/\varepsilon)$ memory and $O(d \log LB/\varepsilon)$ iterations can solve the problem ($\star$) up to error ε.*

1.2 Outline of the Paper

In Section 2, we will briefly review the background material on convex optimization algorithms needed to study Conjectures 1 and 2.

In Section 3, we will attempt to devise new convex optimization algorithms that satisfy the conditions of Conjecture 1. To begin, we will show that the classic ellipsoid algorithm is capable of solving the problem ($\star$) in just $O(d^2 \log LB/\varepsilon)$ iterations while storing only $O(d^2)$ real values in memory, providing strong evidence that Conjecture 1 is true. Having demonstrated that the ellipsoid algorithm both converges in far fewer iterations than gradient descent and requires slightly less memory than the center-of-mass method, we will conclude that Conjecture 1 is likely true. For the remainder of Section 3, we will attempt to improve on Conjecture 1 by exploring some simple modifications to the center-of-mass method and ellipsoid algorithm. The intention behind tweaking these standard algorithms is to devise a variant that slightly improves either the memory usage or oracle complexity, without significantly degrading the other metric.

In Section 4, we will turn our attention to Conjecture 2, which claims a lower bound on the complexity of memory-constrained convex optimization. Although we neither prove nor disprove Conjecture 2, we discuss some interesting and potentially relevant information theoretic techniques used in [1] and [9] to prove similar results in convex optimization. We believe that these tools could plausibly be combined into a proof of Conjecture 2.

2 Background and Notation

As always, a function $F : \mathbb{R}^d \to \mathbb{R}$ is *convex* if

$$F(\alpha x + (1-\alpha)y) \leq \alpha F(x) + (1-\alpha)F(y)$$

for all $x, y \in \mathbb{R}^d$ and $\alpha \in [0,1]$. For a convex function F, the subdifferential of F at $x \in \mathbb{R}^d$ will be denoted by $\partial F(x)$. We will say that F is *L-Lipschitz* provided that

$$|F(x) - F(y)| \leq L\|x - y\|$$

holds for all $x, y \in \mathbb{R}^d$, where $\|\cdot\|$ denotes the ℓ_2 norm.

For our purposes, a *first-order optimization algorithm* for the function F is one that repeatedly makes deterministic queries of the form $x \in \mathbb{R}^d$ to an *oracle*. The oracle answers these queries with the (exact) values of $F(x)$ and an arbitrary subgradient $g \in \partial F(x)$ at x. The number of queries required by an algorithm to find an ε-suboptimal point for F is known as its *oracle complexity* or *query complexity*. For our purposes, we will only consider first-order optimization algorithms whose internal state can be compressed down to a fixed M bits of memory in between consecutive accesses to the oracle.

2.1 Cutting Plane Schemes

The algorithms we will soon consider in our proof of Conjecture 1 are all examples of *cutting plane schemes*. For this reason, we briefly review the relevant properties of cutting plane schemes here.

Fix $x_0 \in \mathbb{R}^d$ and non-zero $g \in \partial f(x_0)$. Let $x^* \in \mathbb{R}^d$ be such that $f(x^*)$ is the solution to ($\star$). The definition of the subgradient yields

$$0 \leq f(x_0) - f(x^*) \leq \langle g, x_t - x^* \rangle \tag{1}$$

In particular, if we define the half space $\mathcal{H} = \{x \in \mathbb{R}^d : \langle g, x_0 - x \rangle \geq 0\}$ then we can guarantee that $x^* \in \mathcal{H}$.

This simple observation inspires an entire class of iterative algorithms based on repeatedly slicing the domain into smaller and smaller feasible regions.

Algorithm 1: General Cutting Plane Scheme, [6] (Section 3.2.6)

```
set E_0 ⊃ B ;
for i = 0, 1, 2, ..., N do
    choose e_i ∈ E_i ;
    query f(e_i) and g_i ∈ ∂f(e_i) ;
    if g_i = 0 then
        return f(e_i) ;
    else
        set H = {x : ⟨g_i, e_i − x⟩ ≥ 0};
        set E_{i+1} ⊃ E_i ∩ H
    end
end
return min(f(e_0), ..., f(e_N));
```

Ideally, the E_i defined in Algorithm 1 will quickly decrease in volume towards x^*. In fact, the E_i must decrease in volume for convergence.

Theorem 2.1. *Let $\mathcal{A}$ be a cutting plane algorithm and let $f : \mathbb{R}^d \to \mathbb{R}$ be L-Lipschitz. Suppose that there exists N large enough such that the set E_N obtained from $\mathcal{A}$ satisfies*

$$\mathrm{Vol}(E_N) < \left(\frac{\varepsilon}{2LB}\right)^d \mathrm{Vol}(B) \tag{2}$$

then $\mathcal{A}$ will find a ε-approximate minimizer for f.

Proof. See Appendix A. □

The size of N in Theorem 2.1 tells us how many iterations it will take minimize f. Clearly, a good cutting plane algorithm will make N small for all choices of f.

2.2 The Center of Mass Method

The center-of-mass method (introduced by [4] and [7] in 1965) is a cutting plane algorithm that gives an explicit way of constructing of the sets E_i used in Algorithm 1. Given E_{i-1}, let c_{i-1} be the *center of mass* of E_{i-1}:

$$c_{i-1} = \frac{\int_{E_{i-1}} x}{\mathrm{Vol}(E_{i-1})} \tag{3}$$

Then we simply define

$$E_i = E_{i-1} \cap \{x \in \mathbb{R}^d : \langle \partial f(c_{i-1}), c_{i-1} - x \rangle \geq 0\} \tag{4}$$

In particular, E_i is the intersection of i half-spaces. The justification for using the center of mass is the following surprising result from geometry:

Lemma 2.2 (Grünbaum (1960), [3])**.** *Let $K \subset \mathbb{R}^d$ be convex. If c is the center of mass of K and $\mathcal{H}$ is any halfspace whose boundary contains c, then*

$$\frac{\mathrm{Vol}(K \cap \mathcal{H})}{\mathrm{Vol}(K)} \leq 1 - \frac{1}{e} \tag{5}$$

Interestingly, this bound is independent of the dimension d. Applying Theorem 2.1, we see that CoM finds an ε-approximate minimizer for *any* f when

$$N > \frac{d \log\left(\frac{\varepsilon}{2LB}\right)}{\log(1 - \frac{1}{e})} \tag{6}$$

In particular, this implies that the query complexity of CoM is $O(d \log\left(\frac{LB}{\varepsilon}\right))$, which is known to be optimal. Moreover, constructing E_i requires knowledge of all previous centers and subgradients, so $O(d^2 \log\left(\frac{LB}{\varepsilon}\right))$ vectors need to be stored in memory. After discretizing to the necessary precision, the total number of bits required is $O(d^2 \log^2\left(\frac{LB}{\varepsilon}\right))$ [12].

3 Toward Upper Bounds

Although the center of mass method obtains optimal oracle complexity, the intersection of half-spaces are too expressive to be saved in subquadratic memory. A natural approach is thus to simplify the sets E_i defined in the center of mass by using an outer approximation. The well known *ellipsoid method* provides slightly simpler simpler approximating sets, and we will show that this is in fact sufficient for solving Conjecture 1. However, it remains unknown if approximations requiring subquadratic memory exist.

3.1 Ellipsoid Method

The ellipsoid method operates by producing a shrinking sequence of ellipsoids $E_0, E_1, \ldots$ that each contain an optimal point x^*. These sets are defined by

$$E_t = \{x \in \mathbb{R}^d : (x - c_t)^T H_t^{-1} (x - c_t) \leq 1\},$$

for $c_t \in \mathbb{R}^n$ and a square matrix $H_t \succeq 0$. The key to the ellipsoid method is the observation from [11] that

$$\mathrm{Vol}(E_{t+1}) \leq \exp\left(-\frac{1}{2d}\right) \mathrm{Vol}(E_0) \tag{7}$$

The matrix H_t can be formed using at most d vectors, which is slightly below the $d \log(1/\varepsilon)$ requirements of the center of mass method. Moreover, the volume decreasing property given in (7) is only marginally worse than Gründbaum's Lemma.

Proposition 3.1. *The ellipsoid method has oracle complexity $O(d^2 \log LB/\varepsilon)$ for the class of convex Lipschitz bounded functions. Moreover, the ellipsoid method requires storing $O(d^2)$ real numbers, strictly less than the center of mass method.*

Proof. Note that after k iterations, the volume of the ellipsoid is E_k is less than $\exp(\frac{-1}{2d})^k \text{vol}(B)$. Thus, after $2d^2 \log(\frac{2LB}{\varepsilon})$ iterations, the volume of E_k is less than $\left(\frac{\varepsilon}{2LB}\right)^d \text{vol}(B)$. Since this bound is independent f, Theorem 2.1 gives $O(d^2 \log(LB/\varepsilon))$ as the oracle complexity.

Clearly, at most d^2 real numbers are required to store the matrix H_t. In fact, only $\frac{d(d-1)}{2} + d$ real numbers are required, although this is still $O(d^2)$. H_t admits an spectral decomposition $U\Gamma U^T$, where $U \in \mathbb{R}^{d\times d}$ is orthonormal and Γ is a diagonal matrix of eigenvalues. The result follows because orthogonal matrices have only $d(d-1)/2$ free parameters.

□

Proposition 2.1 shows that the ellipsoid method has a smaller query complexity than gradient descent and a smaller memory footprint than the center-of-mass method, thus proving Conjecture 1.

Despite this apparent success, the traditional ellipsoid method doesn't offer a completely satisfying solution to the problem of convex optimization with limited memory since its memory usage is still in excess of $\Omega(d^2)$. However, the ellipsoid method does enjoy an $\tilde{O}(d^2)$ rate of convergence. Thus, if only we could reduce the memory footprint required to store the $d \times d$ matrix H_t, we would have achieved a $\text{poly}(d)$ time, $o(d^2)$ space algorithm, resolving a major question in [12]. Although we haven't yet been able to make such a method work, we've at least proved the following:

Proposition 3.2. *For any $t < d$, the ellipsoid algorithm needs to store only $O(td)$ real numbers, and hence $O(td \log LB/\varepsilon)$ bits, in memory across its first t iterations.*

Proof. Consider the update equation for the matrix H_t defining the ellipsoid:

$$H_{t+1} = \frac{d^2}{d^2-1}\left(H_t - \frac{2}{d+1}\frac{H_t w w^T H_t}{w^T H_t w}\right),$$

where w is the subgradient returned by the oracle in iteration t (see [2]). Since H_0 is initialized as a multiple of the identity matrix I_n, a simple induction on t using the above equation and the subadditivity of rank shows that H_t can be expressed as a linear combination of the matrix I_n and another matrix of rank at most t. It follows that H_t can be stored in memory using only $O(td)$ real values. □

Unfortunately, it's not entirely clear how to translate the above observation into a memory-efficient algorithm. Perhaps one could devise an algorithm that involves, say, only $\sqrt{d}$ iterations of the ellipsoid algorithm, combined with other memory-efficient techniques? On the one hand, the above proposition guarantees that such an algorithm would use only $O(d\sqrt{d})$ memory (ignoring dependence on L, B, ε). On the other hand, the volumes of the ellipsoids $\mathcal{E}_t$ decrease very slowly, by a factor of $\exp(-1/2d)$ each iteration, so it's possible that this idea has little to offer.

3.2 Simpler Cutting Plane Schemes

Both the center of mass and ellipsoid methods rely on very specific geometric results about how the volume of these sets decrease after being sliced. Interestingly, Gründbaum's Lemma tells us that the volume decreases the fastest when the sets E_i are the intersection of half-spaces. However, these sets also require unbounded memory (as the number of iterations increase). Ellipsoids require strictly less memory than intersections of half-spaces, yet their volume decreases at a much slower rate. As a result of this observation, we conjecture that there is a relationship between the rate at which the volume of the E_i decreases and the number of vectors required to parameterize E_i.

Unfortunately, it is quite difficult to define E_i using fewer than d vectors while still guaranteeing that $\text{vol}(E_i) \to 0$. Axis aligned rectangles and spheres can be parameterized using only a constant number of $\mathbb{R}^d$ vectors. However, these sets are not expressive enough to satisfy the volume decreasing property of Theorem 2.1. For example, the volume of the minimum bounding axis aligned rectangle will not decrease if the cutting hyperplane passes through diagonal vertices. Of course, it would also suffice to parameterize the E_i with $O(\log(d)$ or $O(\sqrt{d})$ vectors, although it is unclear if any well known classes of sets have this property. Even the $\mathbb{R}^d$ simplex, the polytope with the fewest number of faces, is parameterized by $\Omega(d+1)$ vectors.

An alternate approach is to find a way to efficiently approximate the center of mass c_i. In the case where we can approximate the intersection of half-spaces to an arbitrary precision, the following lemma says that the center of masses will become close.

Lemma 3.3. *Let $A \subset B \subset \mathbb{R}^d$, with $\|x\| \leq C$ for all $x \in B$. If A has center of mass $a \in \mathbb{R}^d$ and B has center of mass $b \in \mathbb{R}^d$, then*

$$\|a - b\| \leq \frac{2C(\mathrm{Vol}(B) - \mathrm{Vol}(A))}{\mathrm{Vol}(B)}.$$

See Appendix B for a proof. It is unlikely that Lemma 3.3 will be useful, however, as approximating the intersection of half-spaces (up to arbitrary precision) with subquadratic memory appears to be impossible.

3.3 Optimizing over Subspaces

In our search for a $\mathrm{poly}(d)$ time, $o(d^2)$ memory convex optimization algorithm, one avenue we considered was to tweak existing convex optimization algorithms that use $\Omega(d^2)$ memory by applying them to slightly lower dimensional subspaces of $\mathbb{R}^d$. Specifically, for a L-Lipschitz and convex function defined F on $\mathbb{R}^d$, let $x^* \in \mathrm{argmin}_{\|x\| \leq B} F(x)$, and let $F^* = F(x^*)$, as usual. We observed the following:

Proposition 3.4. *Let $A \subseteq \mathbb{R}^d$ be an affine subspace of dimension $\sqrt{d}$ and let $F|_A$ denote the restriction of F to A. Then there is an algorithm that finds a point $\hat{x} \in A$ with $F|_A(\hat{x}) - F|_A{}^* \leq \varepsilon$ using $O(d \log^2(LB/\varepsilon))$ bits of memory and $O(\sqrt{d} \log(LB/\varepsilon))$ function and gradient evaluations.*

Proof. Notice that for any points $x, y \in A$ and subgradient $g \in \partial f(x)$, the inner product $\langle g, x - y \rangle$ doesn't change if we replace g with its projection onto the linear subspace of $\mathbb{R}^d$ parallel to A. Moreover, it takes only $O(\sqrt{d} \log(LB/\varepsilon))$ bits of memory to store such a projection (up to the L2 error required by the center-of-mass method). Since the center-of-mass method requires $O(\sqrt{d} \log(LB/\varepsilon))$ iterations to optimize $F|_A$ up to error ε, its overall memory usage is $O(d \log^2(LB/\varepsilon))$. □

We speculate that repeatedly optimizing F over carefully chosen subspaces could lead to a marginal improvement in convergence rate over gradient descent, and the above proposition shows that this could be done with $o(d^2)$ memory. However, we've yet to analyze any such method.

4 Toward Lower Bounds

In the previous sections, most of our work has been spent trying to prove Conjecture 1 by tweaking existing convex optimization algorithms. In this section, we will turn our attention toward potential approaches to proving Conjecture 2, which claims that there is no first-order optimization algorithm for convex Lipschitz bounded functions that runs in just $O(d \log LB/\varepsilon)$ iterations while using only $O(d \log LB/\varepsilon)$ bits of memory. Unlike Conjecture 1, which seeks an improvement over the performance of existing algorithms, Conjecture 2 suggests a *lower bound* on the oracle complexity of convex optimization with limited memory.

4.1 Resisting Oracles

The current state of research indicates at least two promising directions to proceed when proving lower complexity bounds for convex optimization such as Conjecture 2.

The first method often used to prove such lower bounds involves a key tool known as a *resisting oracle*, which we shall briefly describe here. Recall that, during black box first-order convex optimization, the optimization algorithm may only learn about the objective function F by making queries to an *oracle*, which provides the function's value $F(x)$ and a subgradient $g \in \partial F(x)$ at a requested point x. A resisting oracle attempts to answer these queries in the least helpful way possible, with the goal of deteriorating the performance of the optimizer. A resisting oracle may even attempt to confound the optimizer by changing the objective function F to a new function $\tilde{F}$ in the middle of the optimization process, as long as $\tilde{F}$ is compatible with all previous information revealed about F. A number of

theorems on the difficulty of convex optimization, including that the center-of-mass method has optimal query complexity [6], can be proven by cleverly designing resisting oracles.

Unfortunately, it's unclear to what extent we can incorporate the memory constraints of optimizing algorithm into the construction of a resisting oracles. For this reason, this method doesn't seem particularly well-suited for our problem of interest.

4.2 Reduction from Statistical Estimation

An alternative strategy to proving lower bounds for convex optimization which seems more amenable to analyzing memory usage involves the use of information theory to relate convex optimization to statistical estimation. This technique was perhaps first suggested in [1] by Agarwal, Bartlett, Ravikumar, and Wainwright.

The key idea of [1] was to prove the difficulty of optimizing a class of convex functions by leveraging known results on the difficulty of statistical estimation. Specifically, they considered the estimation problem of inferring the *biases* of a list of d biased coins by observing the result of a sequence of independent coin flips. They showed that this problem can be cast as a stochastic convex optimization problem with a *noisy oracle*. Finally, they used information theoretic lower bounds on the number of observations needed for coin-tossing problem to derive similar lower bounds on the complexity of convex optimization.

The main theorem of [1] can be summarized as follows:

Theorem 4.1 ([1])**.** *No first-order stochastic optimization algorithm running for T iterations can guarantee error less than $c\sqrt{d/T}$, for some constant $c > 0$, on the class of all convex functions that are 1-Lipschitz with respect to the ℓ_∞ norm.*

We emphasize that the above theorem pertains to the setting of *noisy oracles* and makes use of the ℓ_∞ norm. Both of these traits differ significantly from the setup of our problem of interest (see Section 2), especially in high dimensions. Nevertheless, it will be instructive to discuss, at a high level, the proof of Theorem 4.1 before explaining how it could be adapted to analyze memory-constrained optimization algorithms.

Proof Outline. The first step in the proof of 4.1 is to construct a suitable "difficult" subclass of convex 1-Lipschitz (w.r.t. ℓ_∞) functions to be optimized. These functions, denoted $\mathcal{G} = \{g_\alpha : \alpha \in \mathcal{V}\}$ are parameterized by vertices $\mathcal{V} \subseteq \{-1, +1\}^d$ of a d-dimensional hypercube, as follows:

$$g_\alpha(x) = \frac{1}{d}\sum_{i=1}^{d}\left[\left(\frac{1}{2} + \alpha_i\delta\right)\left|x_i + \frac{1}{2}\right| + \left(\frac{1}{2} - \alpha_i\delta\right)\left|x_i - \frac{1}{2}\right|\right]$$

(here δ is a small positive constant). The quantities $\frac{1}{2} \pm \alpha_i\delta$ appearing in the above expression should be interpreted as the probabilities that a biased coin lands either heads or tails. Indeed, this observation allows [1] to construct a *noisy* oracle for functions in $\mathcal{G}$ as follows. When queried at $x \in \mathbb{R}^d$:

- The oracle first picks an index $i \in \{1, \ldots, d\}$ uniformly at random.
- Then, it draws $b_i \in \{0, 1\}$ according to a Bernoulli distribution with parameter $1/2 + \alpha_i\delta$.
- Finally, it returns the value and sub-gradient of the function

  $$\hat{g}_\alpha(x) = b_i\left|x + \frac{1}{2}\right| + (1 - b)\left|x - \frac{1}{2}\right|,$$

  which agree, *in expectation*, with the value and sub-gradient of the function g_α at x.

The above noisy oracle connects the problem of estimating the bias of a coin to that of optimizing a convex function. With this in mind, [1] choose the vertices of $\mathcal{V}$ so that, for distinct $\alpha, \beta \in \mathcal{V}$, the minima of the corresponding functions g_α and g_β are "far apart" (the exact discrepancy measured used can be found in [1]). They then show that any convex optimization algorithm for the class $\mathcal{G}$

that attains a low error *must* be able to distinguish between bias vectors α and β and hence solve the coin-flipping problem. Finally, they use standard information theoretic tools like Fano's inequality to show that distinguishing α from β would require observing a large number of coin flips. Thus, optimizing the convex 1-Lipschitz functions in $\mathcal{G}$ must require a large number of oracle queries. $\square$

As stated, Theorem 4.1 doesn't make any special consideration for *memory-constrained* convex optimization algorithms. However, we suspect that with some effort, the proof of Theorem 4.1 can be adapted to a lower bound on the memory needed for convex optimization by reducing from a slightly different statistical estimation problem, studied by Shamir in [9]:

Definition 4.2 (Hide-and-Seek Problem [9])**.** *Fix $j \in \{1, \ldots, d\}$ and $0 < \delta < 1/2$. Consider a random vector $\mathbf{x} \sim \{0,1\}^d$ in which each coordinate $\mathbf{x}_i$ is an independent Bernoulli random variable with parameter $1/2 + \delta \mathbf{1}_{i=j}$. Given an i.i.d. sample of instances generated from the distribution of $\mathbf{x}$, detect j.*

It was shown in [9] that the hide-and-seek problem is one that becomes exponentially harder when memory constraints are imposed. Thus, we suspect that modifying the proof of Theorem 4.1 to use the hide-and-seek problem for the reduction to convex optimization could result in a lower bound on not only the number of iterations needed to optimize the subclass $\mathcal{G}$ of convex functions, but also on the amount of *memory* required.

At the moment, the obstacles we face in carrying out such a proof include embedding the hide-and-seek problem as a stochastic convex optimization problem in such a way that distinct $j, j' \in \{1, \ldots, d\}$ correspond to sufficiently "different" optimization objectives. For the sake of Conjecture 2, a more serious issue is that the methodology of [1] fundamentally relies on the presence of noise in the oracle, while our problem of interested from [12] does not allow this.

5 Challenges & Next Steps

A challenge that we've faced in our research stems from the fact that our open problem, as formulated by Srebro and Woodworth, deals with arbitray L-Lipschitz continuous functions F, which may not necessarily be smooth. Although this seemed to be a minor technicality at first, we now believe that the potential nonexistance of the derivative ∇F is precisely what makes this open problem challenging and interesting.

Indeed, if we assume that F is β-smooth (so that ∇F is β-Lipschitz) and F is α-strongly continuous, then an algorithm like Nesterov's accelerated gradient descent (analyzed in [2]) can locate an ε-suboptimal point x^* for F using only $O(\sqrt{\beta/\alpha}\log(B^2/\varepsilon))$ first-order oracle queries with the same space complexity as traditional gradient descent. Unfortunately, this result, along with a wide variety of modern analyses of smooth convex optimization algorithms (e.g. [5]), does not shed light on our question of interest, which concerns first-order convex optimization algorithms that work for *all* bounded L-Lipschitz functions.

Despite this challenge, we believe that our work in Section 3 could provide the beginnings of a construction of a faster limited-memory convex optimization algorithm. We also believe that the information theoretic ideas presented in Section 4 can ultimately be leveraged to prove a lower bound on the complexity of memory-constrained optimization. In the future, we hope to explore these ideas further.

References

[1] Alekh Agarwal, Peter L. Bartlett, Pradeep Ravikumar, and Martin J. Wainwright. Information-theoretic lower bounds on the oracle complexity of stochastic convex optimization. *IEEE Transactions on Information Theory*, 58:3235–3249, 2010.

[2] Sébastien Bubeck. Convex optimization: Algorithms and complexity. *Foundations and Trends in Machine Learning*, 8:231–357, 2014.

[3] B. Grünbaum. Partitions of mass-distributions and of convex bodies by hyperplanes. *Pacific J. Math.*, 10(4):1257–1261, 1960.

[4] A. Y. Levin. On an algorithm for the minimization of convex functions over convex functions. *Soviet Mathematics Doklady*, 1965.

[5] Philipp Moritz, Robert Nishihara, and Michael I. Jordan. A linearly-convergent stochastic L-BFGS algorithm. In *AISTATS*, 2015.

[6] Yurii Nesterov. *Introductory Lectures on Convex Optimization: A Basic Course*. Springer Publishing Company, Incorporated, 1 edition, 2014.

[7] D. J. Newman. Location of the maximum on unimodal surfaces. *J. ACM*, 1965.

[8] Ran Raz. Fast learning requires good memory: A time-space lower bound for parity learning. *J. ACM*, 66:3:1–3:18, 2018.

[9] Ohad Shamir. Fundamental limits of online and distributed algorithms for statistical learning and estimation. In *NIPS*, 2013.

[10] Vatsal Sharan, Aaron Sidford, and Gregory Valiant. Memory-sample tradeoffs for linear regression with small error. *ArXiv*, abs/1904.08544, 2019.

[11] N. Shor and N. Zhurbeko. The minimization method using space dilation in the direction of difference of two sequential gradients. *Kibernetika*, 1971.

[12] Blake Woodworth and Nathan Srebro. Open problem: The oracle complexity of convex optimization with limited memory. In Alina Beygelzimer and Daniel Hsu, editors, *Proceedings of the Thirty-Second Conference on Learning Theory*, volume 99 of *Proceedings of Machine Learning Research*, pages 3202–3210, Phoenix, USA, 25–28 Jun 2019. PMLR.

A Proof of Theorem 2.1

Theorem 2.1. *Let $\mathcal{A}$ be a cutting plane algorithm and let $f : \mathbb{R}^d \to \mathbb{R}$ be L-Lipschitz. Suppose that there exists N large enough such that the set E_N obtained from $\mathcal{A}$ satisfies*

$$\mathrm{Vol}(E_N) < \left(\frac{\varepsilon}{2LB}\right)^d \mathrm{Vol}(B) \tag{8}$$

then $\mathcal{A}$ will find a ε-approximate minimizer for f.

Proof. The proof is a simple generalization of Theorem 3 from [12]. Set $\alpha = \left(\frac{\varepsilon}{2LB}\right)$ and define $B_\alpha = \{\alpha x + (1-\alpha)x^* : x \in B\}$ so that $\mathrm{Vol}(B_\alpha) = \alpha^d \mathrm{Vol}(B)$. If $y \in B_\alpha$ the convexity of f implies

$$f(y) = f(\alpha x + (1-\alpha)x^*) \leq \alpha f(x) + (1-\alpha) f(x^*) \tag{9}$$

Furthermore, if $g \in \partial f(x)$, then $f(x) \leq \langle g, x - x^* \rangle + f(x^*)$. Applying Cauchy-Schwarz, we obtain

$$f(x) \leq ||g|| \cdot ||x - x^*|| + f(x^*) \tag{10}$$

Putting (10) into (9), we obtain

$$f(y) \leq \alpha(||g|| \cdot ||x - x^*|| + f(x^*)) + (1-\alpha) f(x^*) \tag{11}$$

A simple application of Cauchy-Schwarz yields $||g|| \leq L$ and $||x - x^*|| \leq 2B$, so (11) becomes

$$f(y) \leq \alpha(LB + f(x^*)) + (1-\alpha) f(x^*) = f(x^*) + 2\alpha LB \tag{12}$$

using the definition of α, we get $f(y) \leq f(x^*) + \varepsilon$, so y is an ε-accurate minimizer.

Take N large enough so that $\mathrm{Vol}(E_N) < \alpha^d \mathrm{Vol}(B)$. Since $x^* \in E_k$ for every k, it follows that there is a $k < N$ such that $y \in B_\alpha \cap E_k$ and $y \notin E_{k+1}$. For this k, let e_k be the queried point and let $g_k \in \partial f(e_k)$ be the subderivative. Then

$$f(e_k) \leq f(y) + \langle g, e_k - y \rangle \tag{13}$$

But y was cut off by E_{k+1}, so $\langle g, e_k - y \rangle \leq 0$. Thus, $f(e_k) \leq f(y)$ and $f(e_k)$ is also an ε-accurate minimizer.

□

B Proof of Lemma 3.3

Lemma 3.3. *Let $A \subset B \subset \mathbb{R}^d$, with $\|x\| \leq C$ for all $x \in B$. If A has center of mass $a \in \mathbb{R}^d$ and B has center of mass $b \in \mathbb{R}^d$, then*

$$\|a - b\| \leq \frac{2C(\mathrm{Vol}(B) - \mathrm{Vol}(A))}{\mathrm{Vol}(B)}.$$

Proof. Expanding the definition of the center of mass yields

$$\begin{aligned}
a - b &= \frac{\int_A x\,dx}{\mathrm{Vol}(A)} - \frac{\int_B x\,dx}{\mathrm{Vol}(B)} \\
&= \frac{1}{\mathrm{Vol}(A)\mathrm{Vol}(B)} \left(\mathrm{Vol}(B) \int_A x\,dx - \mathrm{Vol}(A) \int_B x\,dx \right) \\
&= \frac{1}{\mathrm{Vol}(A)\mathrm{Vol}(B)} \left(\mathrm{Vol}(B-A) \int_A x\,dx + \mathrm{Vol}(A) \int_A x\,dx - \mathrm{Vol}(A) \int_A x\,dx - \mathrm{Vol}(A) \int_{B-A} x\,dx \right) \\
&= \frac{1}{\mathrm{Vol}(A)\mathrm{Vol}(B)} \left(\mathrm{Vol}(B-A) \int_A x\,dx - \mathrm{Vol}(A) \int_{B-A} x\,dx \right).
\end{aligned}$$

Therefore,

$$\begin{aligned}
\|a - b\| &= \frac{1}{\mathrm{Vol}(A)\mathrm{Vol}(B)} \left\| \mathrm{Vol}(B-A) \int_A x\,dx - \mathrm{Vol}(A) \int_{B-A} x\,dx \right\| \\
&\leq \frac{\mathrm{Vol}(B-A)}{\mathrm{Vol}(A)\mathrm{Vol}(B)} \int_A \|x\|\,dx + \frac{1}{\mathrm{Vol}(B)} \int_{B-A} \|x\|\,dx \\
&\leq \frac{C\mathrm{Vol}(B-A)}{\mathrm{Vol}(B)} + \frac{C\mathrm{Vol}(B-A)}{\mathrm{Vol}(B)}.
\end{aligned}$$

□